

Envoy — Planning Doc

Strategy, parallel build plan, demo, cut decisions.

For: Rushil · From: Rayan · Re: Voice Agents Hackathon (Cekura × Daily × NVIDIA × AWS × Twilio), May 30
Doc 2 of 4 · built on the official starter: github.com/pipecat-ai/yc-voice-agents-hackathon

TL;DR

Envoy is a voice-first autonomous agent that does real things over the phone (calls a business, negotiates, books, writes a real Calendar event + email) and can't be manipulated into betraying you — and when a new attack slips through, it writes its own guardrail, verifies the fix, and never falls for it again. We build it on the official Pipecat starter, reuse its flower-shop bot as the restaurant counterpart, run the brain on NVIDIA Nemotron, and drive the self-heal loop with Cekura. **The build is structured so Rayan and Rushil never edit the same file.**

The four docs

- **Description** — what Envoy is, plain English.
- **Planning** (this one) — strategy, the parallel build, demo, cut decisions.
- **Prompts** — literal copy-pasteable Claude Code prompts per track, grounded in the actual repo.
- **Submission** — copy-paste text for the form.

1. Context: the starter, the stack, the brief

The organizers ship a working Pipecat starter: a single **inbound** bot (“Field & Flower,” a flower shop that takes delivery orders), with all backends mocked, in two flavors — a GPT-4.1 build and an NVIDIA Nemotron build. We build on it.

The actual stack (corrected — it’s not Riva)

Piece	What we use
LLM (brain)	NVIDIA Nemotron 3 Super 120B — open-weights, hosted on AWS; endpoints provided at the start of the day (no self-hosting).
STT	NVIDIA Nemotron Speech Streaming (V2). Gradium STT is the V1 fallback.
TTS	Gradium.
Orchestration	Pipecat. Transports: SmallWebRTC (local dev) + Twilio (phone).
Eval / auto-improve	Cekura via its Claude Code plugin (MCP + skills); provider = Pipecat.
Deploy	Pipecat Cloud. AWS = storage for the guardrail/eval flywheel + the Nemotron brain runs there.

Two things the starter changes in our favor

- 1. The flower bot IS our restaurant.** That inbound order-taker is the exact shape of the counterpart Envoy calls — reskin it into a reservation line + add hostile mode, and the counterpart agent is most of the way done.
- 2. No GPU to stand up.** The Nemotron endpoints are provided. The old “open-weights setup eats the day” risk is gone — hour 0 is just “paste the endpoint and confirm it talks.” (Trade-off: our AWS story is thinner, so we put the flywheel storage on AWS and note the brain runs there.)

The brief, mapped to Envoy (say this in the write-up)

Brief stage	What Envoy does
Build & Customize	NVIDIA Nemotron open-weights brain on AWS, policy-tuned for constraint-holding negotiation; Nemotron speech-streaming STT.
Deploy at Scale	Pipecat over Twilio PSTN; streaming, barge-in, sub-second turns shown on the dashboard.
Simulate & Evaluate	Cekura adversarial + functional scenarios via the Claude Code plugin; LLM-judged per turn.
Auto-Improve	The self-heal loop: a Cekura-flagged failure → new guardrail → replay → verified blocked → versioned. The centerpiece.

Judging signal worth banking: the organizers explicitly say they want great examples of *using Cekura to improve performance* and *using the open-source NVIDIA models*. Our self-heal loop is the former; the Nemotron brain is the latter. Make both loud.

2. The idea + the trap

Lead with: “An AI that gets things done for you over the phone — and can’t be manipulated into betraying you.” **Earn at the end:** “The chief of staff for the agentic internet.”

The one trap: looking like “Cekura but worse”

Cekura already tests agents and does red-teaming. Three rules: (1) Envoy is an agent that **does things**, not a tool that tests agents — hardening is the trust feature, never the product. (2) We **use** Cekura (power user), we don’t compete. (3) Lead with what it does; don’t say “pen test for voice” — that’s their line.

Restaurant demo, general scope. Keep Envoy vertical-agnostic — the parser, guardrails, negotiation, and self-heal are all general; the *only* restaurant-specific thing is the demo goal (“book dinner under \$60”). The pitch is “the agent you can trust to act on the phone,” shown booking dinner.

3. The build — designed so we never touch the same file

The whole plan is built around zero merge conflicts. One short blocking phase, then fully parallel tracks with single-owner directories. The only crossings are one import boundary and two state files with different readers/writers.

Phase 0 — Rayan only (~1 hr, the only sequential part)

Rayan lays the foundation everything imports; Rushil writes pure content in parallel (no shared files).

- **Rayan builds:** fork the starter + confirm *bot-nemotron.py* runs locally on the provided endpoints; *contracts.py* (data shapes + file paths, locked); *llm_client.py* (a thin Nemotron text client both tracks import); a *pipeline_factory*; stub modules for brain/counterpart/selfheal/dashboards so imports resolve; empty *data/guardrails.json*, *eval_history.json*, *events.jsonl*. Push.
- **Rushil writes (in parallel, imports nothing yet):** the restaurant persona prompt, the attack library, the G0 guardrail seed, and the Cekura scenario list — all as content in his own new files.

The parallelism unlock

llm_client.py lets Rushil run and test the ENTIRE intelligence + self-heal story in **text** (brain + counterpart + monitor + patcher + replay + variant) while Rayan builds the voice/telephony/integration platform. They share one brain module and converge only at the voice step (Slice 3).

Ownership map (single-owner directories)

Rayan — the pipe + platform	Rushil — the brain + proof + win
<i>contracts.py</i> (sole editor), <i>llm_client.py</i> , <i>pipeline_factory</i> ; <i>envoy/pipeline</i> + <i>run_envoy</i> ; <i>telephony/</i> (outbound Twilio); <i>integrations/</i> (Calendar, Gmail + tool registration); <i>infra/</i> (AWS storage, deploy); all config (<i>pyproject</i> , <i>.env</i> , <i>pcc-deploy.toml</i> , <i>README</i>).	<i>brain/</i> (system prompt, guardrail seed, constraint parser, negotiation, <i>build_messages</i> , tool schemas); <i>counterpart/</i> (reskinned flower bot + normal/hard/hostile modes); <i>selfheal/</i> (monitor, patcher, loop, variant); <i>dashboards/</i> (3 panels); <i>evals/</i> (Cekura scenarios + <i>/cekura-report</i> runs).

The crossing points (the only shared surface)

Seam	Type	Writer to Reader
<i>contracts.py</i>	code (locked)	Rayan edits → both import. Change only via a 5-min huddle.
<i>llm_client.py</i>	code	Rayan → both import (the text-mode LLM call).
brain interface	code	Rushil defines exports → Rayan’s pipeline imports them.
<i>data/guardrails.json</i>	file	Rushil’s patcher writes → Rayan’s pipeline reads each turn.
<i>data/events.jsonl</i>	file	Rayan’s pipeline writes → Rushil’s dashboards read.

Conflict rule: if you need something in the other person’s file or in *contracts.py*, post in chat — don’t edit it. Because directories are single-owner, you can both push to main constantly with no conflicts.

4. Vertical slices (each shippable; ownership tagged)

- **Phase 0 (Rayan, ~1 hr, blocking).** Foundation above. Rushil drafts content in parallel.
- **Slice 1 — text core (parallel).** Rushil: brain (parser + prompt + G0 + negotiation) + counterpart persona; run Envoy-vs-counterpart in text via *llm_client*. **CHECKPOINT ~12:00 — does Envoy hold the line in text?** Rayan: Envoy voice pipeline (wrap the brain in Pipecat + Nemotron + Gradium over WebRTC) — talk to it in the browser.
- **Slice 2 — (parallel).** Rushil: the self-heal loop (monitor via Cekura → patcher writes *guardrails.json* → replay → variant), all in text. Rayan: Calendar + Gmail integrations + tool registration — real side effects on a successful run.

- **Slice 3 — voice convergence (the one pairing seam, ~30 min together).** Rayan: outbound Twilio dial; Envoy calls the counterpart bot's number (or a real restaurant). Rushil: deploy the counterpart bot. This is the only tight integration point — schedule it.
- **Slice 4 — (parallel).** Rayan: latency, AWS flywheel storage, Pipecat Cloud deploy, live link. Rushil: the three dashboards (read events.jsonl + guardrails.json + eval_history.json), the Cekura eval view, record segments.
- **Slice 5 — integrate + record + submit.**

Cut lines

Minimum winning submission = Phase 0 + Slices 1–2 + real side effects + the self-heal recorded. The text end-to-end story (acts → manipulated → self-heals → blocked) is complete on its own; voice is the upgrade. Cut first: Cekura eval dashboard polish, extra attack types, latency micro-tuning, the live link.

5. The demo (pre-recorded, 60 seconds — hard cap)

60 seconds must show all three rubric pillars + both judging signals. The real call carries utility + technical; the self-heal flash carries creativity + auto-improve (Cekura) + NVIDIA. Captions throughout (muted first pass). The README carries the full depth.

Time	Beat	What the judge sees/hears	~t
0:00–0:05	Hook + dial	Caption: "Envoy is calling a real restaurant to book your table." Ring; "Hello, [restaurant]..."	5s
0:05–0:30	Real call (utility + technical)	Envoy talks to a real human, negotiates within budget, books. Hard-cut to a real Calendar event + email landing. Caption: "Real restaurant. Real human. Real booking — no human approved it."	25s
0:30–0:52	Self-heal flash (creativity + auto-improve)	A counterpart tries to extract a card number/login → BREACH → a guardrail writes itself → replay → BLOCKED → reworded variant → also BLOCKED → trust score ticks up. Caption: "It can't be social-engineered — verified by Cekura, reasoning on NVIDIA Nemotron."	22s
0:52–1:00	Close	One line + the name + "full system in the README."	8s

Discipline: record several real-call takes, keep the cleanest ~25s; the controlled counterpart is the shake-proof backup. Render the self-heal flash from the dashboard (fully controllable). The scripted self-heal example (attack, R1, variant) lives in the prompts doc §4.

6. Open decisions

- **Name:** Envoy (alts Aegis, Sentinel) — lock in 5 min.
- **Real-call target:** Rushil lines up one consented restaurant; the controlled counterpart bot is the always-safe backup. California is all-party consent and it's going in a public video — get written consent or skin the brand.
- **Format:** submissions 6 PM, informal demos 6–8 PM, judges present 8 PM. Pre-recorded video is the artifact — confirm at check-in.

- **Latency target:** pick a sub-second turn number to show on the dashboard; confirm what the Nemotron path actually hits in Slice 1.

7. Hour-by-hour (9 AM start, 6 PM submission)

Time	Rayan (pipe + platform)	Rushil (brain + proof + win)
9–10	PHASE 0 (blocking): fork, confirm Nemotron bot runs, contracts.py + llm_client + factory + stubs + data files. Push.	Draft persona, attack library, G0 seed, Cekura scenarios (no shared files).
10–12	Envoy voice pipeline over WebRTC (wrap the brain).	Brain (parser+prompt+G0+negotiate) + counterpart; run Envoy-vs-counterpart in text. CHECKPOINT @ 12.
12–2	Calendar + Gmail integrations + tool registration; real side effects.	Self-heal loop in text: monitor (Cekura) + patcher → guardrails.json + replay + variant.
2–3:30	Outbound Twilio dial. PAIR @ ~2:00 for voice convergence.	Deploy counterpart bot. PAIR for convergence, then start dashboards.
3:30–4:30	Latency, AWS flywheel storage, Pipecat Cloud deploy, live link.	Three dashboards; record ready segments.
4:30–5:15	Repo/README cleanup; on-call for re-takes.	Pre-record the self-heal + variant; assemble.
5:15–6	Final cleanup + secrets scrub.	Record/edit video, Devpost, submit with buffer.

Sync points

~12:00 does the text agent hold the line? ~2:00 voice-convergence pairing (the only tight seam) + cut decision (if behind, lock the text-core story and pre-record now) ~4:30 end-to-end walk; polish lock.

The demo is the artifact. Build backwards from it. Single-owner directories + one import boundary + two state files = no merge conflicts.